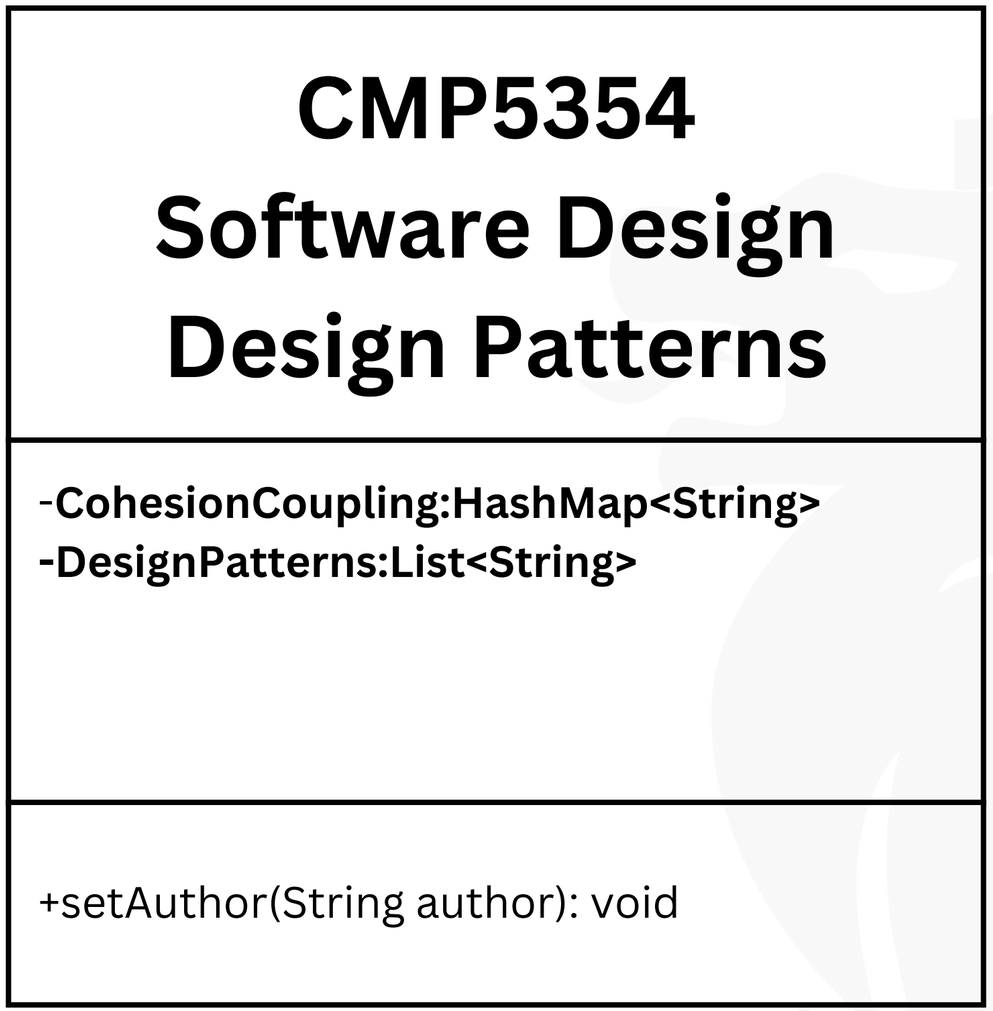
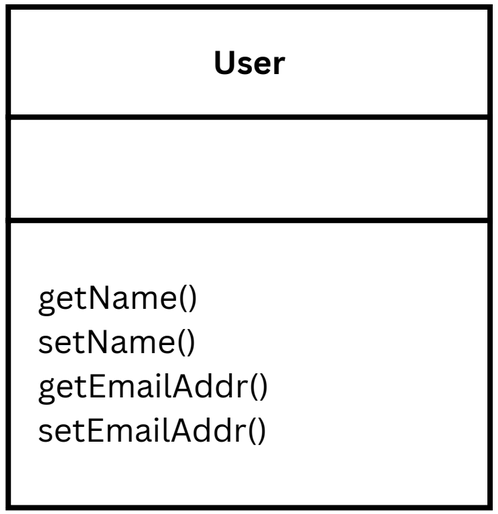

CMP5354
Software Design
Design Patterns
-CohesionCoupling:HashMap<String>
-DesignPatterns:List<String>
+setAuthor(String author): void

Cohesion

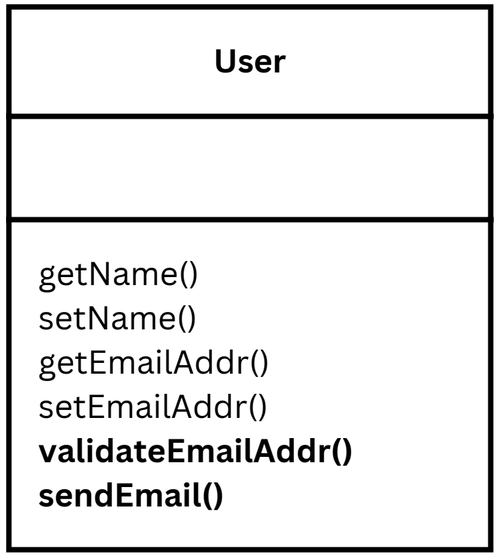
- Cohesion is the degree to which the **elements** inside a **class** belong together.



High Cohesion

A class with high cohesion contains elements that are **tightly related** to each other and united in their purpose.

For example, all the methods within a User class should represent the user behavior.



Low Cohesion

A class is said to have low cohesion if it contains **unrelated** elements.

For example, a User class containing a method on how to validate the email address. User class can be responsible for storing the email address of the user but not for validating it or sending an email

Coupling

- Coupling is the degree of **interdependence** between software classes.

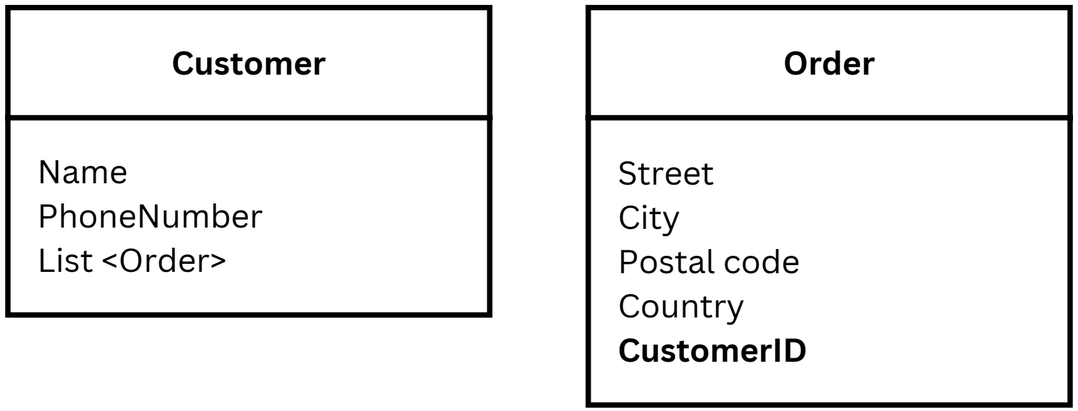
- Coupling is about how changing one thing requires change in another.

Tight Coupling

Two classes have high coupling (or tight coupling) if they are closely connected.

For example, two concrete classes storing references to each other and calling each other's methods.

Customer and Order are tightly coupled to each other. The Customer is storing the list of all the orders placed by a customer, whereas the Order is storing the reference to the Customer object.



Loose Coupling

Every time the customer places a new order, we need to add it to the order list present inside the Customer.

Order only needs to know the customer identifier and does need a reference to the Customer object. We could make the two classes loosely coupled.

Classes with low coupling among them work mostly independently of each other.

Comparison

1

Cohesion and coupling are related to each other.
Each can affect the level of the other.

eg1

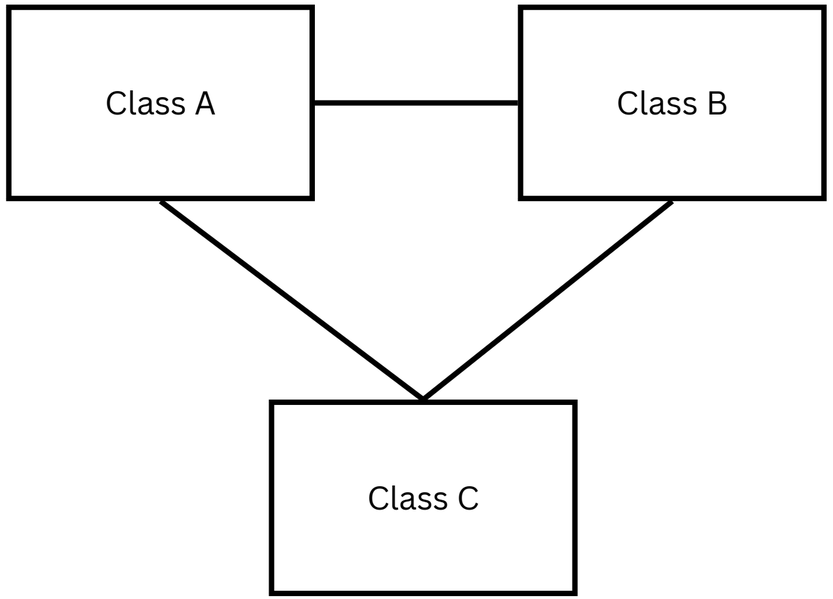
High cohesion correlates with loose coupling

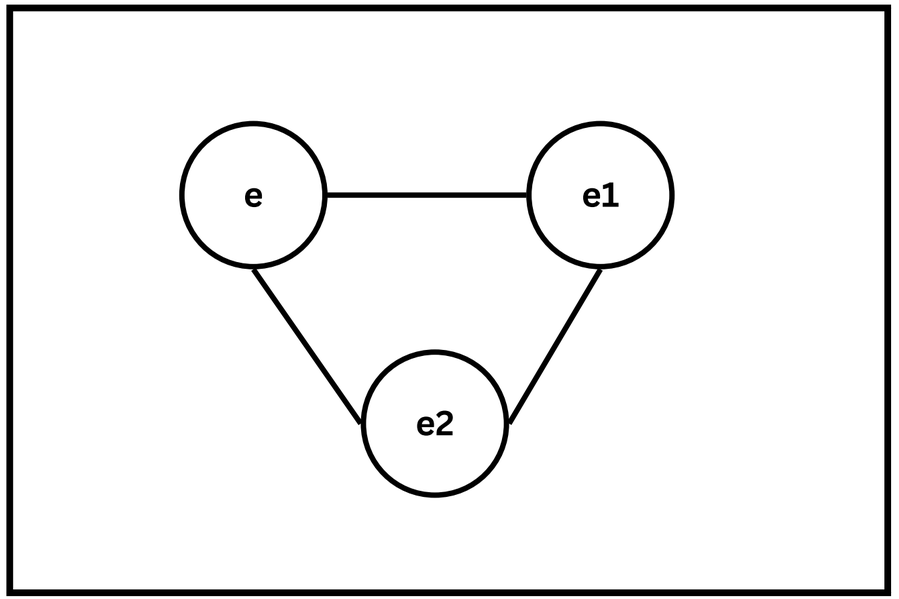
A class having its elements tightly related to each other and serving a single purpose would sparingly interact and depend on other modules. Thus, will have loose coupling with other classes.

eg2

Low cohesion correlates with tight coupling

Classes could be heavily dependent on each other due to the elements spread across the two classes. And thus, will have low cohesion.





Coupling

Cohesion

Coupling deals with the **interdependence** between software classes.
Cohesion deals with the **interconnection** between the elements of the same class.

Cohesion	Coupling
Cohesion is the degree to which the elements inside a class belong together	Coupling is the degree of interdependence between the class
A class with high cohesion contains elements that tightly related to each other and united in their purpose	Two classes might have high coupling (or tight coupling) if they are closely connected and dependant of each other
A class is said to have low cohesion if it contains unrelated elements	Classes with low coupling among them work mostly independently of each other
High cohesive classes reflect higher quality of software design	Loose coupling reflects the higher quality of software design

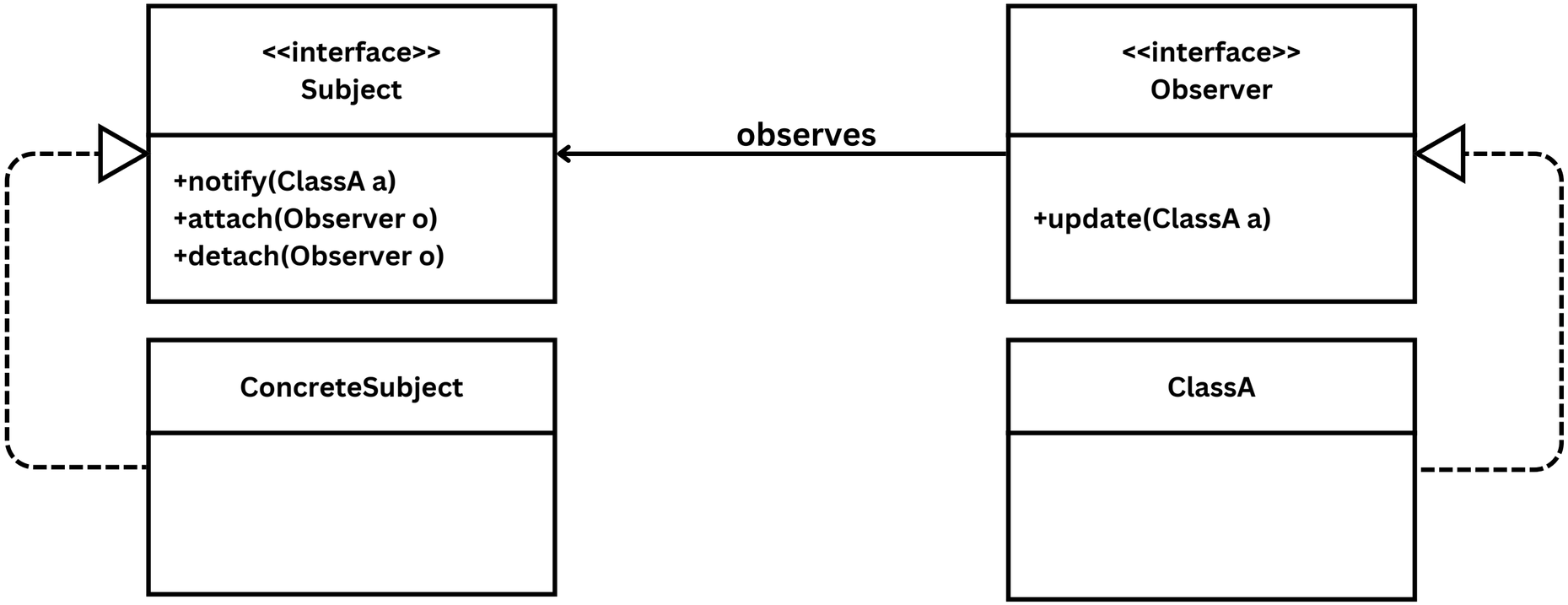
Design Patterns

Design Patterns

Each pattern describes a problem that occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice.

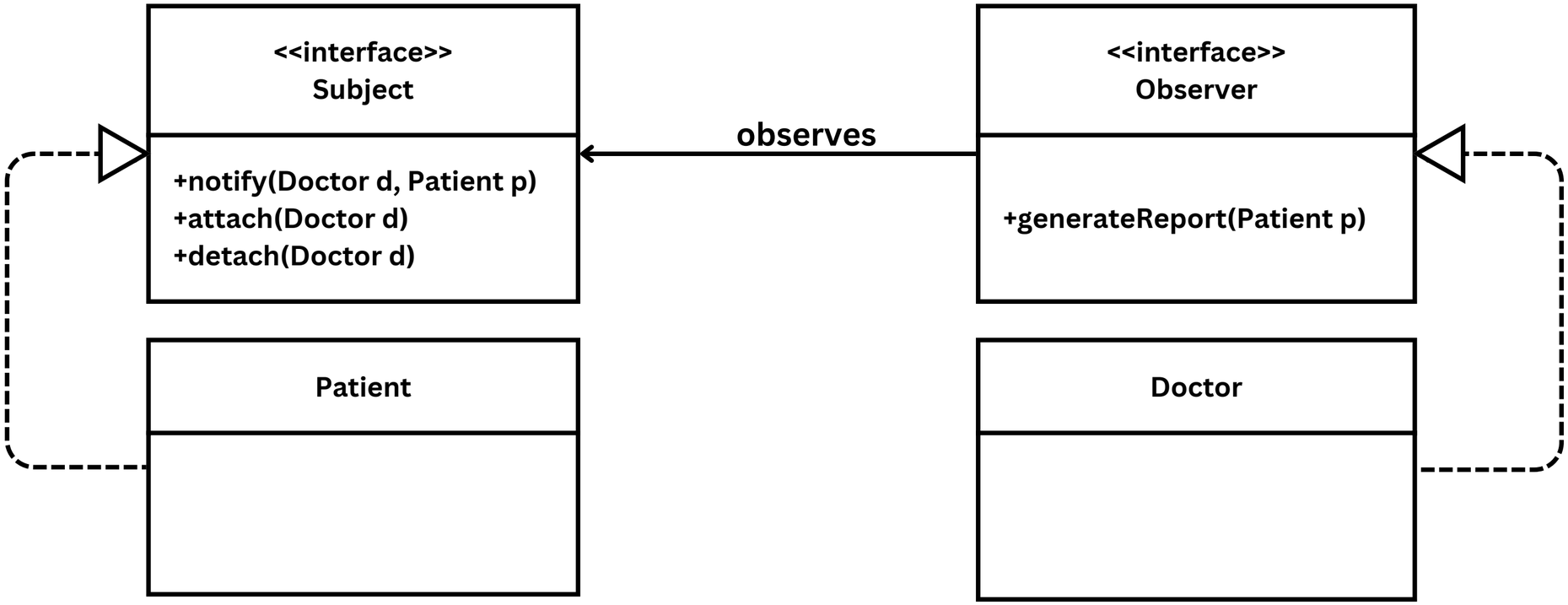
Observer/Publish-Subscribe Pattern

When a certain change happens in the subject class, all its observers (0-many classes) are automatically notified and updated, ensuring consistency across the system.
The notify() and update() can have different names and parameters based on the case study.



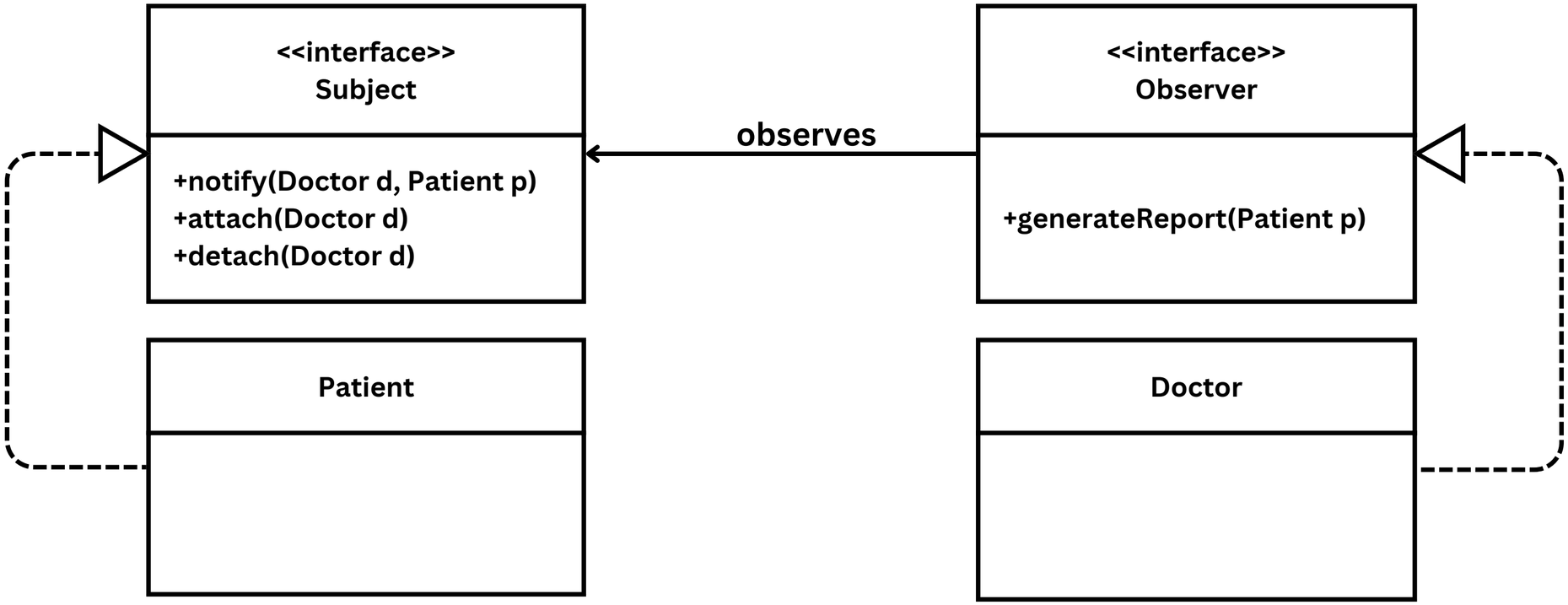
Observer/Publish-Subscribe Pattern Example

A doctor needs to be notified every time a new patient is registered to his record. When the doctor is notified, a report is generated for that patient's medical history.



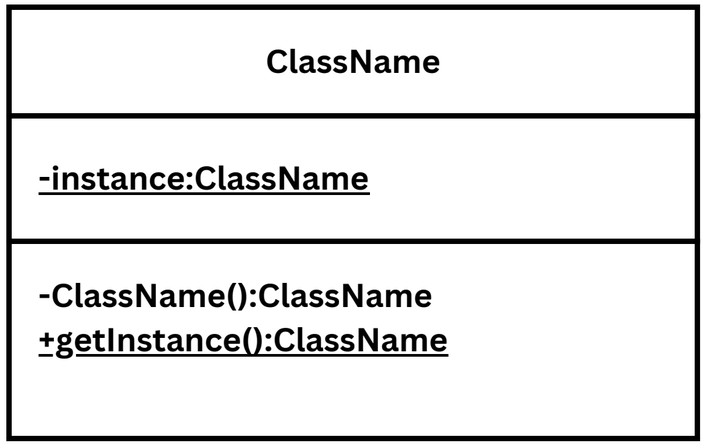
Observer/Publish-Subscribe Pattern Example

In implementation, all the methods in the interfaces have to be implemented in the Patient and Doctor. The generateReport() is the equivalent of the update() method in the earlier slide. The naming of the update() method can be adjusted to suit the case study. Within the notify() method, the generateReport() method needs to be called.



Singleton Pattern

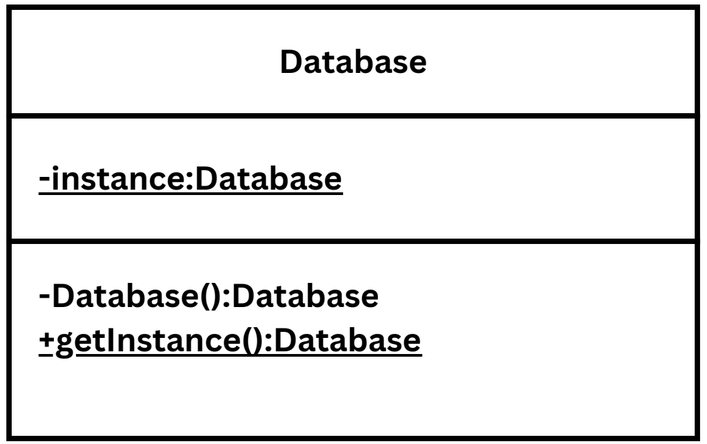
It ensures that a class has only one instance, while providing a global point of access to that instance.
Underlined attributes and methods are static. They are not specific to an instance of the class.




```java
public class ClassName{

    private static ClassName instance = null;

    private ClassName()
    {
      //add constructor content
    }

    public static ClassName getInstance(){
        if (instance == null)
            instance = new ClassName();

        return instance;
}
```

Singleton Pattern Example

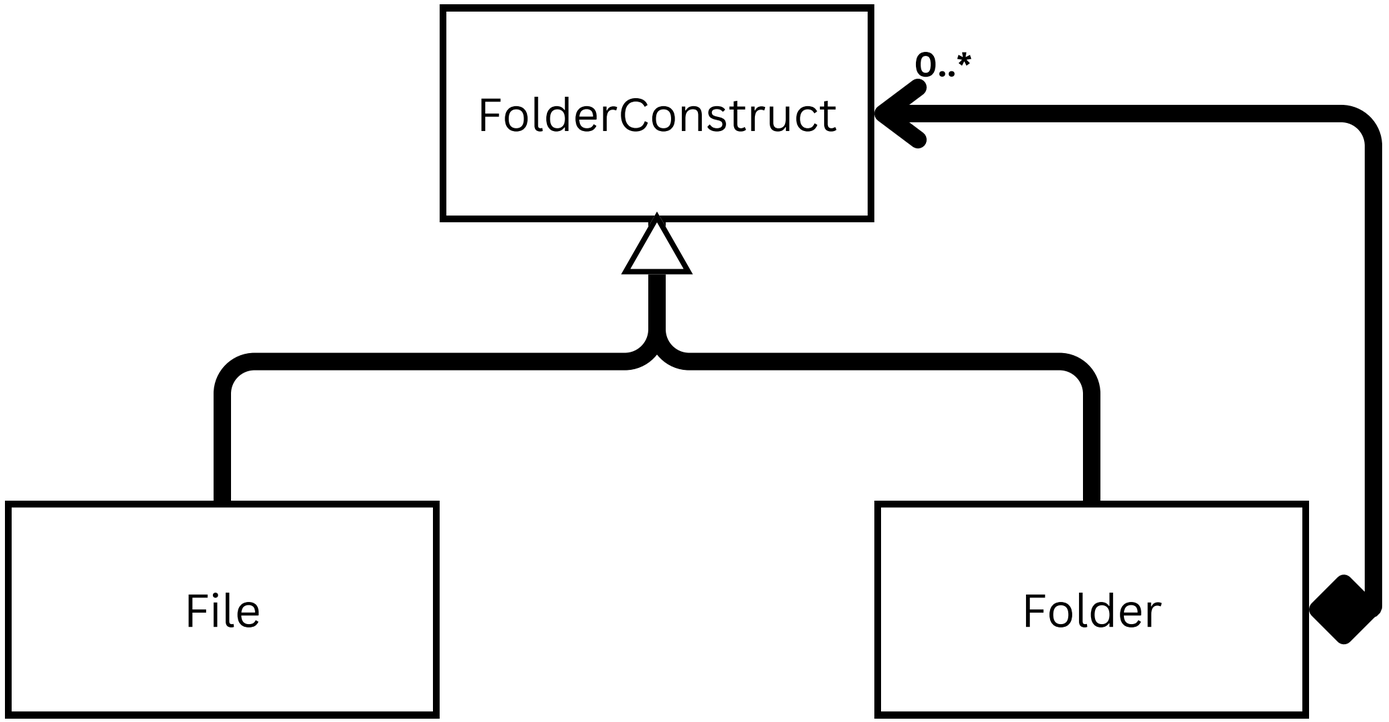
A database is only to be created once in the system to keep it consistent with all updates.




```java
public class Database{

    private static Database instance = null;

    private Database()
    {
      //add constructor content
    }

    public static Database getInstance(){
        if (instance == null)
            instance = new Database();

        return instance;
    }
}
```

Composition Pattern and Example

It allows you to compose objects into tree structures to represent part-whole relationships. It lets clients treat individual objects and compositions of objects uniformly.



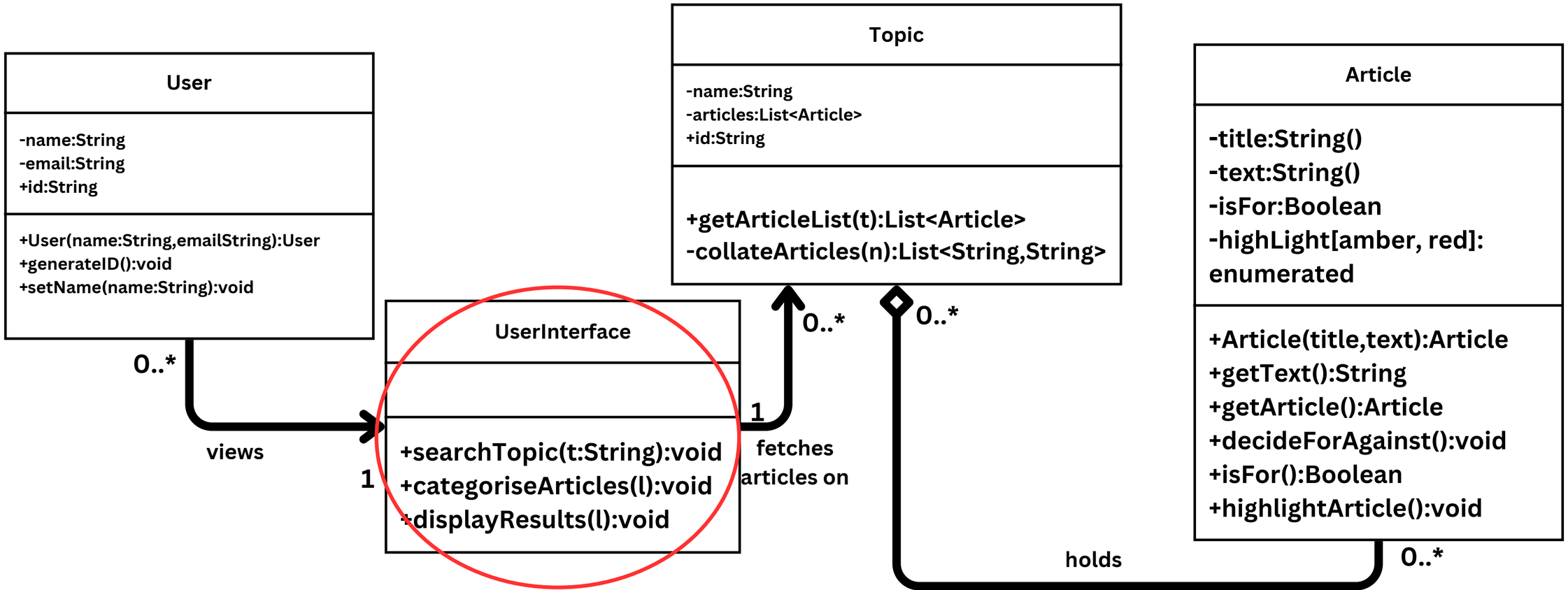
```java
// Abstract class to represent both Files and Folders
public abstract class FolderConstruct {
    protected String name;

    public FolderConstruct(String name) {
        this.name = name;
    }
```

```java
public class File extends FolderConstruct {

    public File(String name) {
        super(name);  // Call the constructor of FolderConstruct
    }

    // Implement the display method
    @Override
    public void display(String indent) {
        System.out.println(indent + "File: " + name);
    }
}
```

```java
public class Folder extends FolderConstruct {
    private List<FolderConstruct> contents;  // Composition: A Fo

    public Folder(String name) {
        super(name);
        contents = new ArrayList<>();  // Initialize the contents
    }

    // Method to add a file or folder (composition)
    public void add(FolderConstruct item) {
        contents.add(item);
    }
```

Facade Pattern and Example

It provides a simplified interface to a complex system, making it easier to use by hiding the complexities and exposing only the essential functions through a unified interface.



```java
public class UserInterface {
    // Attribute: Holds the currently searched Topic object
    private Topic currentTopic;

    // Constructor
    public UserInterface() {
        this.currentTopic = null; // No topic initially
    }
```

```java
public void searchTopic(String t) {
    Topic topic = new Topic(t);
    List<Article> articles = topic.getArticleList(t);
    categoriseArticles(articles);
    displayResults(articles);
}

public void categoriseArticles(List<Article> articles) {
    int forCounter = 0;
    int againstCounter = 0;

    for (Article a : articles) {
        a.decideForAgainst();
        a.highlightArticle();

        if (a.isFor()) {
            forCounter++;
        } else {
            againstCounter++;
        }
    }

    if (forCounter == 0 || againstCounter == 0) {
        System.out.println("Articles are not balanced, cannot be displayed");
        return;
    }
}
```

MVP without Facade Pattern - Search Topic

Screen A – Search Topic (searchTopic(t:String):void)

User can search, but cannot categorise articles or view results yet

searchTopic(t:String):void

Topic: Enter topic text ...

Date range: Last 30 days

Region: Global

Search Topic | Next -> Categorise Screen

Problem:

- User cannot see any articles yet
- User cannot mark For / Against / Neutral here
- User cannot highlight 'amber' or 'red'
- User must remember search context when moving to next screen

This is what it looks like if searchTopic() lives in its own separate interface.
No categoriseArticles() / displayResults() access here.

MVP without Facade Pattern - Categorise Articles

Screen B – Categorise Articles (categoriseArticles(1):void)

User tries to classify stance and highlight, but cannot change topic or see full results table

categoriseArticles(1):void

Rules:

- Decide if article is 'For'
- Decide if 'Against'
- Or 'Neutral'

☐ Show For

☐ Show Neutral

☐ Show Against

Highlight: [amber] [red]

[<- Back to Search Screen] [Next -> Results Screen]

This is what it looks like if categoriseArticles() is isolated in its own interface.
No searchTopic() controls, and still no final displayResults().

Problem:

- User does not see live article list here
- User cannot confirm stance assignments actually make sense in context
- User cannot edit topic, date range, etc. from this screen
- Must bounce between screens to iterate

MVP without Facade Pattern - Display Results

Screen C – Display Results (displayResults(l):void)

User can view table of results, but cannot change topic, filters, stance, or highlight here

displayResults(l):void Results for: "Vaccine Mandates in Universities"

Sort by: Credibility (High -> Low) Export

Headline / Snippet	Publisher	Stance	Highlight
Article 1 headline... Short summary text snippet...	Publisher X	For / Neutral / Against	amber
Article 2 headline... Short summary text snippet...	Publisher X	For / Neutral / Against	amber
Article 3 headline... Short summary text snippet...	Publisher X	For / Neutral / Against	amber
Article 4 headline... Short summary text snippet...	Publisher X	For / Neutral / Against	amber
Article 5 headline... Short summary text snippet...	Publisher X	For / Neutral / Against	amber

Why am I seeing this? (explain stance & credibility logic)

Problem:

- You can see results, but you cannot:
  - * change topic or time range
  - * re-categorise For/Neutral/Against
  - * change highlight amber/red
- User must jump back to other screens to adjust anything.

This is what it looks like if displayResults() is isolated by itself.
All context and controls live on different pages.

MVP with Facade Pattern - All in One